

НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО

Факультет программной инженерии и компьютерной техники

Системы ввода/вывода

Лабораторная работа №1

**«Принципы организации ввода/вывода без операционной
системы»**

Вариант 2

Выполнили:

Ибрагимова Айгуль

Брель Мария

Преподаватель:

Табунщик Сергей Михайлович

Санкт-Петербург

2026

Содержание

1	Цель	3
2	Задачи	4
3	Основная часть	5
3.1	Описание функций putchar и getchar	5
3.1.1	Функция putchar	5
3.1.2	Функция getchar	5
3.2	Интерфейс вызова функций OpenSBI	5
3.2.1	Структура sbiret	5
3.2.2	Функция sbi_call	6
3.3	Описание функций OpenSBI варианта задания	6
3.3.1	Get SBI implementation version	6
3.3.2	Hart get status	7
3.3.3	Hart stop	7
3.3.4	System Shutdown	8
4	Результаты работы программы	9
4.1	Скриншоты вывода консоли	9

Цель

Изучение принципов организации ввода/вывода в системах без операционной системы на примере архитектуры RISC-V с использованием интерфейса OpenSBI. Получение практических навыков разработки низкоуровневых функций ввода-вывода и взаимодействия с аппаратными ресурсами через стандартизированный интерфейс SBI (Supervisor Binary Interface).

Задачи

1. Реализовать функцию putchar вывода данных в консоль
2. Реализовать функцию getchar для получения данных из консоли
3. На базе реализованных функций putchar и getchar написать программу, позволяющую вызывать определенные варианты функции OpenSBI посредством взаимодействия пользователя через меню
4. Запустить программу и выполнить вызов пунктов меню, получив результаты их работы
5. Оформить отчет по работе в электронном формате

Функции Open SBI

1. Get SBI implementation version
2. Hart get status (должно быть возможно задавать номер ядра)
3. Hart stop
4. System Shutdown

Основная часть

Описание функций putchar и getchar

3.1.1 Функция putchar

Функция `putchar` предназначена для вывода одного символа на консоль. В контексте разработки ядра без операционной системы эта функция реализована через вызов интерфейса OpenSBI.

Реализация функции выглядит следующим образом:

```
1 void putchar(char ch) {
2     sbi_call(ch, 0, 0, 0, 0, 0, 0, SBI_CONSOLE_PUTCHAR);
3 }
```

Функция принимает один параметр — символ `ch`, который необходимо вывести. Внутри функция вызывает `sbi_call` с идентификатором расширения `SBI_CONSOLE_PUTCHAR` (0x01), передавая символ в качестве первого аргумента. Остальные аргументы заполняются нулями, так как они не используются данной функцией SBI.

3.1.2 Функция getchar

Функция `getchar` предназначена для чтения одного символа с консоли. Она также реализована через интерфейс OpenSBI.

Реализация функции:

```
1 int getchar(void) {
2     struct sbiret ret;
3
4     do {
5         ret = sbi_call(0, 0, 0, 0, 0, 0, 0, SBI_CONSOLE_GETCHAR);
6     } while (ret.error == SBI_ERR_FAILED);
7
8     return (int) ret.error;
9 }
```

Функция выполняет циклический опрос консоли через `sbi_call` с идентификатором расширения `SBI_CONSOLE_GETCHAR` (0x02). Если символ пока не доступен, функция SBI возвращает код ошибки `SBI_ERR_FAILED` (-1), и опрос продолжается. Когда символ становится доступным, он возвращается в поле `error` структуры `sbiret`, которое затем приводится к типу `int` и возвращается как результат функции.

Интерфейс вызова функций OpenSBI

OpenSBI (Open Source Supervisor Binary Interface) — это открытая реализация интерфейса SBI для архитектуры RISC-V. SBI предоставляет стандартизированный набор сервисов для взаимодействия между режимом супервизора (S-mode) и машинным режимом (M-mode).

3.2.1 Структура sbiret

Результат вызова функций SBI возвращается в виде структуры `sbiret`:

```

1 struct sbiret {
2     long error; // Код ошибки или возвращаемое значение
3     long value; // Дополнительное возвращаемое значение
4 };

```

Поле `error` содержит код возврата функции. Значение 0 означает успешное выполнение, отрицательные значения — коды ошибок (например, `SBI_ERR_FAILED = -1`). Поле `value` используется для передачи дополнительных данных в зависимости от вызываемой функции.

3.2.2 Функция `sbi_call`

Функция `sbi_call` является универсальным интерфейсом для вызова сервисов OpenSBI из режима супервизора:

```

1 struct sbiret sbi_call(long arg0, long arg1, long arg2, long arg3,
2                       long arg4, long arg5, long fid, long eid) {
3     register long a0 __asm__ ("a0") = arg0;
4     register long a1 __asm__ ("a1") = arg1;
5     register long a2 __asm__ ("a2") = arg2;
6     register long a3 __asm__ ("a3") = arg3;
7     register long a4 __asm__ ("a4") = arg4;
8     register long a5 __asm__ ("a5") = arg5;
9     register long a6 __asm__ ("a6") = fid;
10    register long a7 __asm__ ("a7") = eid;
11
12    __asm__ __volatile__ ("ecall"
13                          : "=r"(a0), "=r"(a1)
14                          : "r"(a0), "r"(a1), "r"(a2), "r"(a3),
15                          "r"(a4), "r"(a5), "r"(a6), "r"(a7)
16                          : "memory");
17    return (struct sbiret){.error = a0, .value = a1};
18 }

```

Параметры функции:

- `arg0`-`arg5` — аргументы для вызываемой функции SBI
- `fid` (Function ID) — идентификатор функции внутри расширения
- `eid` (Extension ID) — идентификатор расширения SBI

Функция размещает аргументы в соответствующие регистры архитектуры RISC-V (`a0`-`a7`) и выполняет инструкцию `ecall`, которая вызывает переход в машинный режим. После возврата из обработчика значения регистров `a0` и `a1` упаковываются в структуру `sbiret` и возвращаются вызывающему коду.

Описание функций OpenSBI варианта задания

В рамках данной лабораторной работы реализованы вызовы следующих функций OpenSBI:

3.3.1 Get SBI implementation version

Функция получения версии реализации SBI использует расширение Base Extension (`EID = 0x10`) и функцию с идентификатором `FID = 2`.

```

1 struct sbiret result = sbi_call(0, 0, 0, 0, 0, 0, 2, 0x10);
2 printf("SBI implementation version: %d.%d\n",
3       (result.value >> 16) & 0xFFFF,
4       result.value & 0xFFFF);

```

Функция не требует входных параметров (все аргументы установлены в 0). Результат возвращается в поле `value` структуры `sbiret`, где старшие 16 бит содержат мажорную версию, а младшие 16 бит — минорную версию реализации SBI.

Константы:

- `SBI_EXT_BASE = 0x10` — идентификатор базового расширения
- `Function ID = 2` — функция получения версии реализации

3.3.2 Hart get status

Функция получения статуса аппаратного потока (`hart`) использует расширение Hart State Management (HSM) Extension (`EID = 0x48534D`) с идентификатором функции `FID = 2`.

```

1 // Пользователь вводит номер ядра
2 int hart_id = getchar() - '0';
3
4 struct sbiret res = sbi_call(hart_id, 0, 0, 0, 0, 0, 2, 0x48534D);
5
6 if (res.error != 0) {
7     put("Hart status error\n");
8 } else {
9     printf("Hart status: %d\n", res.value);
10 }

```

Функция принимает в качестве первого аргумента (`arg0`) идентификатор аппаратного потока, статус которого необходимо получить. В случае успеха (`error == 0`) поле `value` содержит текущий статус `hart`:

- 0 — `STARTED` (запущен)
- 1 — `STOPPED` (остановлен)
- 2 — `START_PENDING` (ожидает запуска)
- 3 — `STOP_PENDING` (ожидает остановки)

Константы:

- `SBI_EXT_HSM = 0x48534D` — идентификатор расширения управления состоянием `hart`
- `Function ID = 2` — функция получения статуса `hart`

3.3.3 Hart stop

Функция остановки текущего аппаратного потока использует расширение HSM Extension (`EID = 0x48534D`) с идентификатором функции `FID = 1`.

```

1 void sbi_hart_stop(void) {
2     sbi_call(0, 0, 0, 0, 0, 0, 1, 0x48534D);
3     while(1); // Бесконечный цикл на случай возврата
4 }

```

Функция не принимает аргументов и останавливает текущий hart. После успешного выполнения контроль не должен вернуться в вызывающий код, однако для безопасности добавлен бесконечный цикл.

Константы:

- SBI_EXT_HSM = 0x48534D — идентификатор расширения управления состоянием hart
- Function ID = 1 — функция остановки hart

3.3.4 System Shutdown

Функция завершения работы системы использует расширение System Reset Extension (EID = 0x53525354) с идентификатором функции FID = 0.

```
1 void sbi_system_shutdown(void) {  
2     sbi_call(0, 0, 0, 0, 0, 0, 0, 0x53525354);  
3     while(1); // Бесконечный цикл на случай возврата  
4 }
```

Функция выполняет корректное завершение работы системы. Первый аргумент (0) указывает тип сброса — SHUTDOWN (выключение). После успешного выполнения система должна быть выключена, однако для безопасности добавлен бесконечный цикл.

Константы:

- SBI_EXT_SRST = 0x53525354 — идентификатор расширения сброса системы
- Function ID = 0 — функция сброса/выключения
- Reset type = 0 — SHUTDOWN (выключение системы)

Choose an option:

1. Get SBI implementation version
2. Hart get status
3. Hart stop
4. System Shutdown

SBI implementation version: 1.7

Enter hart ID: 0

Hart status: 1

Enter hart ID: 7

Hart status error

Stopping system%